

AI CODE IN-THE-WILD SECURITY RISK RESEARCH REPORT

• AI 生成代码在野安全风险研究报告

In-the-Wild Analysis & Vulnerability Assessment
真实场景分析与漏洞评估

报告范围 • REPORT SCOPE //

AI 代码生成 • 安全风险 • 趋势洞察 • 漏洞分析

AI Code Generation & Security Risks Trends & Vulnerability Analysis

日期 • DATE //

2025 年 12 月

December 2025

联合发布 • JOINTLY RELEASED BY //



腾讯安全平台部
悟空代码安全团队



北京大学
Narwhal-Lab



复旦大学
系统软件与安全实验室
威胁情报小组

摘要

随着大型语言模型 (LLM) 驱动的 AI 代码生成技术不断深入软件开发全生命周期，其在提升研发效率的同时，也重塑了软件供应链的安全边界。本报告系统评估了当前 AI 代码生成技术的能力现状、潜在风险及其演进趋势。我们构建了从 IDE 辅助工具到自主 AI 软件工程师的分层能力体系，并分析了由代码幻觉、知识截断和训练数据偏差引发的漏洞注入风险、数据泄露隐患与合规挑战。基于对开源生态的实证研究，我们进一步揭示了 AI 在漏洞生命周期中的双重角色：既能提升修复效率，也可能成为新缺陷的源头。研究表明，随着 AI 生成代码比例的上升，开发者角色正从“代码编写者”转向“审查者与验证者”。本报告最终提出构建全生命周期的安全治理框架，以确保在持续集成环境下实现安全可靠的 AI 辅助开发。

关键词：AI 代码生成；软件供应链安全；代码幻觉；漏洞分析；代码安全治理

目录

1 概述	4
2 AI 代码生成能力和应用概况	5
2.1 核心能力分类	5
2.2 主流 AI 代码生成产品与平台	5
2.3 小结	6
3 AI 生成代码的风险与挑战	7
3.1 技术特性与内在矛盾	7
3.2 直接安全风险	8
3.3 间接与合规性风险	8
3.4 小结	9
4 AI 生成代码趋势与特点	10
4.1 时间发展趋势	10
4.2 编程语言分布	11
4.3 小结	13
5 AI 生成代码的安全风险与漏洞特征	15
5.1 漏洞生命周期中 AI 的角色演变	15
5.2 AI 引入漏洞的特征画像	16
5.3 小结	18
6 风险缓解建议	19
6.1 建立多维度的评测基准	19
6.2 增强模型本体安全性	19
6.3 人机协同治理	20
6.4 小结	20
7 总结	21

1 概述

随着大型语言模型（LLM）技术的飞跃式发展，人工智能（AI）已深度融入软件开发的各个环节，其中 AI 代码生成技术正引发一场深刻的范式革命。从 GitHub Copilot、Cursor 等集成开发环境（IDE）到 Devin 等新兴的自主 AI 软件工程师，这些工具通过自动化代码编写、调试、测试乃至项目部署，极大地提升了开发效率与创新速度。然而，这场技术革新是一柄双刃剑。在享受自动化带来的便利时，一个前所未有且日益严峻的安全挑战也随之浮现：AI 生成的代码是否安全可靠？

当 AI 成为软件供应链中一个新兴的、自动化的“代码贡献者”时，其内在的技术特性、训练数据的偏差以及与开发者交互的模式，都可能引入传统软件安全体系未曾充分考虑的新型风险。这些风险不仅限于传统的漏洞注入，更延伸至软件供应链安全、知识产权合规、乃至开发团队安全文化等多个维度。因此，对 AI 代码生成技术的安全影响进行系统性、深层次的分析，已成为学术界与工业界刻不容缓的重要议题。

本报告聚焦于 AI 代码生成技术的安全威胁态势。报告首先界定该技术的能力边界与典型应用模式，为风险评估构建基准。随后，从生成机制层面剖析代码特性，详细论述由其衍生的多维度安全风险。报告通过量化分析开源生态中 AI 生成代码的渗透趋势及其在漏洞生命周期中的演变规律，旨在为理解 AI 代码生成的实际安全效能提供数据支撑。

2 AI 代码生成能力和应用概况

2.1 核心能力分类

人工智能驱动的代码生成技术，特别是基于 Transformer 架构的大型语言模型 (LLM)，已从单一的代码补全工具演变为涵盖软件开发全生命周期的智能化生态系统。为系统性地评估其在安全与效能方面的影响，我们将现有技术能力依据其在软件开发生命周期中的作用范围及抽象层次，归纳为三个主要层级：微观级辅助、宏观级自主构建以及特定领域生成。

在微观层面，AI 最广泛的应用形态是深度集成于集成开发环境 (IDE) 的实时辅助功能。这类能力将传统开发工具转变为主动的编程协作伙伴，其核心是智能代码补全，它实现了从语法级提示到语义级生成的范式转变，能够基于项目全局上下文生成代码块。代码解释与文档生成旨在弥合自然语言与编程语言的鸿沟，提升软件的可维护性。同时，在实时调试与错误修复方面，AI 模型可在编码阶段进行静态分析，主动识别潜在缺陷并对错误提供修复建议，这种“即时反馈”机制有效地推动了“安全左移”开发理念的落地。

在宏观层面，随着 AI 智能体 (AI Agent) 技术的兴起，AI 的角色正从辅助者向系统设计者与执行者转变。这一层级超越了简单的片段生成，具备了项目级脚手架构建与全流程自动化的能力。例如，Devin 和开源项目 OpenHands 展示了 AI 在长期规划与执行方面的潜力。它们不仅能根据自然语言描述自动化生成项目所需的配置文件与基础架构，更能独立完成从需求分析、代码编写到错误调试的完整闭环。核心的原型与功能生成能力支持将抽象的业务逻辑直接转化为包含前端交互、后端逻辑及数据持久层的可运行代码，极大地加速了从概念验证到最小可行性产品的迭代周期。

除了通用编程语言，AI 在处理领域特定语言 (DSL) 方面也表现出色。其中，数据库查询生成尤为成熟，AI 模型具备模式感知能力，可将复杂的自然语言数据检索需求转化为精确、高效的 SQL 查询语句。同样，在系统集成与自动化运维 (DevOps) 领域，AI 能够根据需求生成符合特定规范的 API 调用代码或功能正确的命令行脚本，例如为 Kubernetes 生成管理指令，显著提升了云原生环境下的运维效率。

2.2 主流 AI 代码生成产品与平台

随着底层 LLM 技术的飞速发展，AI 代码生成已演变为一个繁荣且多样化的技术生态。市场上的相关产品与平台在形态、功能和集成深度上各具特色，共同构成了当前的技术版图。这些产品可被划分为四种主流范式：IDE/编辑器集成插件、对话式 AI、一体化开发平台以及新兴的 AI 软件工程师代理，如表 1 所示。

IDE/编辑器集成插件是目前最成熟、最广泛应用的工具形态。此类产品以扩展插件方式嵌入现有开发环境，在不中断开发者心流的前提下提供细粒度代码补全与建议。GitHub Copilot、CodeBuddy、支持私有模型微调的 Tabnine，以及具备内置安全扫描

表 1: 主流 AI 代码生成工具分类与代表产品

分类范式	代表性产品与平台
IDE/编辑器集成插件	GitHub Copilot, CodeBuddy, Tabnine, Amazon CodeWhisperer, JetBrains AI Assistant, 通义灵码等
对话式 AI 与代码生成	OpenAI ChatGPT, Google Gemini, Anthropic Claude, Grok, Phind, Perplexity AI 等
一体化开发平台	Cursor, Codex, Claude Code, GitHub Codespaces, Qoder, Trae, Antigravity 等
AI 软件工程师	Devin, Devika, OpenHands, Aider, Sweep 等

能力的 Amazon CodeWhisperer 等是该范式的典型代表。

对话式 AI 平台（如 ChatGPT、Gemini、Claude 等）通过自然语言界面支持更高层次的编程协助，包括问题分析、方案推演与多步任务分解，已成为开发者的重要技术咨询入口。一体化开发平台进一步增强了这种能力，通过将 AI 深度整合到编写、调试、重构、运行与协作的完整开发流程中，实现“AI 原生”的软件工程体验。Cursor、Claude Code 和 GitHub Codespaces 等是当前较具代表性的实践。

AI 软件工程师则代表了该领域的前沿探索趋势，目标是从工具式辅助迈向自主完成任务的智能体。以 Cognition AI 发布的 Devin 等为代表的系统已经能够进行任务规划、代码实现、调试验证直至简单部署等端到端流程，并催生了 OpenHands、Devika、Aider、Sweep 等多种开源或轻量级替代方案。这一方向预示软件工程范式可能发生进一步的自动化与自治式变革。

2.3 小结

本章从能力结构与产品生态两方面对当前 AI 代码生成技术进行了系统整理。AI 已具备从局部代码补全、解释与修复，到项目级脚手架构建与自动化开发，再到面向数据库与运维场景的领域特定生成等多种功能，逐步覆盖软件开发生命周期中的主要环节。这些能力通过 IDE 插件、对话式接口、一体化开发环境以及 AI 软件工程师代理等不同形态落地，形成了从轻量集成到高度自治的多层次技术体系。

AI 代码生成正在从辅助性工具向具备自主执行能力的开发实体演进，产品形态也开始呈现分层化与专业化趋势。这一演进显著提高了研发效率，但也引入了新的安全、可靠性与过程可控性问题。本章的分析为后续的风险评估与检测方案设计奠定了必要的技术基础与生态背景。

3 AI 生成代码的风险与挑战

AI 代码生成技术在重塑软件开发流程的同时，也引入了一系列独特的内在特性与多维度的伴生挑战。深入剖析这些特性，是评估其对软件安全性、代码质量及开发文化深远影响的前提。这些挑战并非孤立的技术缺陷，而是贯穿于模型训练、人机交互、法律合规以及开发者行为模式的系统性问题。图1 展示了横跨技术、法律与组织层面的风险分类架构。

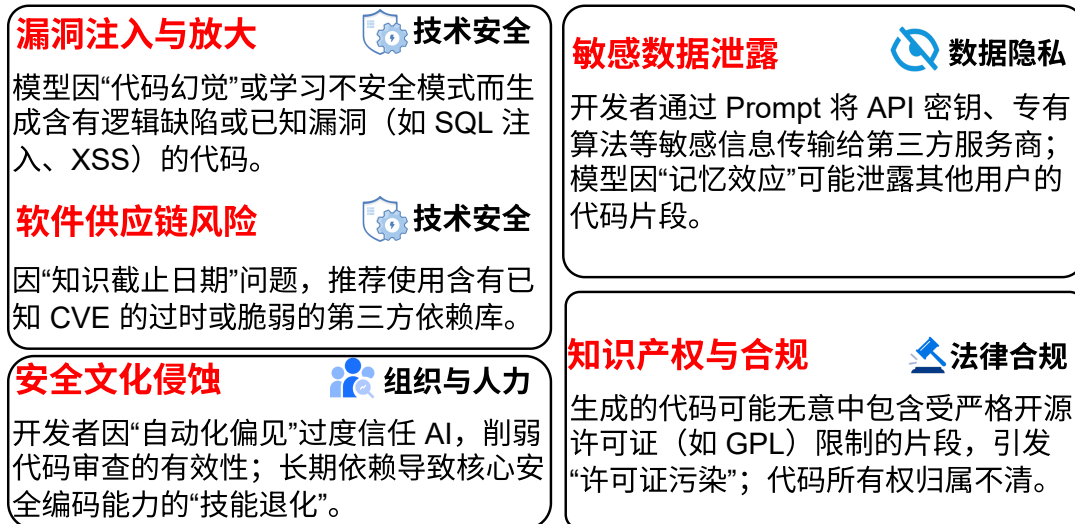


图 1: AI 代码主要生成风险分类

3.1 技术特性与内在矛盾

AI 代码生成技术的核心优势在于通过自动生成样板代码、算法实现与测试用例，显著提升软件开发生命周期的效率。其效能根植于先进的上下文感知能力——现代 AI 模型能够综合分析抽象语法树、项目级依赖乃至全库编码模式，从而从单纯的代码补全工具演进为具备逻辑推理能力的编程辅助系统。

然而，这种能力呈现出显著的内在矛盾：

- **规范性与碎片化的冲突：**经特定代码库微调的模型可成为推行编码规范的有力工具；反之，未经定制的通用模型因其训练数据的异构性，可能在项目中引入不一致的编码风格与架构模式，从而损害代码库的长期可维护性。
- **提示敏感性：**输出质量与输入提示的质量呈高度正相关。模糊或存在歧义的自然语言描述极易导致模型生成功能偏离甚至错误的代码。这要求开发者掌握精确的需求分析与逻辑抽象能力（即提示工程）。若团队该能力参差不齐，反复调试劣质代码的成本可能抵消自动化的效率红利。

3.2 直接安全风险

尽管能力强大，AI 代码生成技术当前最严峻的挑战之一是其固有的可靠性问题，这直接转化为多种安全风险。

首要风险是“代码幻觉”导致的漏洞注入。该现象指模型以极高的“自信度”生成语法正确但语义上存在严重逻辑缺陷的代码，易引发开发者的自动化偏见。这些缺陷往往直接映射为严重的安全漏洞。

案例分析：已有研究发现，当要求 AI 工具（如早期版本的 GitHub Copilot）实现加密功能时，模型可能优先生成使用已被弃用且存在严重安全缺陷的 DES 算法或 ECB 工作模式的代码，因为它在训练数据中出现的频率可能高于更安全现代的 AES-GCM 模式。

另一项重大直接风险源于模型的“知识截止日期”。这一特性导致模型对最新的安全公告一无所知，可能在其生成的代码中推荐使用已知包含严重通用漏洞披露（CVEs）的第三方库。

案例分析：在关键漏洞如 Log4j 的“Log4Shell”（CVE-2021-44228）或 OpenSSL 的“Heartbleed”被披露后，一个知识库未更新的模型仍会继续生成使用脆弱版本依赖项的配置和代码，相当于在软件供应链中主动引入了已知攻击向量。

与 AI 服务的交互过程本身也构成了数据泄露风险。开发者在提示中包含的任何敏感信息，如 API 密钥或专有算法，都可能被传输并存储于第三方服务器。模型的“记忆效应”也可能导致其在生成代码时，无意中复现训练数据中属于其他用户的独特代码片段或敏感信息，构成跨用户的数据泄露。

3.3 间接与合规性风险

AI 生成代码的归属权模糊性给 IP 管理和开源合规带来挑战。模型的训练数据源自数百万个遵循不同许可证的开源项目，若模型生成了一段实质上源自于 GPL 等强传染性许可证的代码，而开发者将其用于商业闭源项目中，则可能触发“许可证污染”，给企业带来巨大的法律和商业风险。

案例分析：这一风险并非纯理论，针对 GitHub Copilot 的集体诉讼案（J. Doe 1 v. GitHub, Inc.）的核心争议之一，即是该工具是否在未经授权和归属的情况下，复现了受严格开源许可证保护的代码片段，从而对下游用户构成潜在的合规风险。代码所有权的模糊性也可能在企业并购或技术尽职调查中构成障碍。

除了法律与合规层面，AI 工具的广泛应用还可能对开发团队的安全文化产生长期的、更为隐蔽的负面影响。开发者的“自动化偏见”可能导致其盲目信任 AI 生成的结果，从而显著降低代码审查的深度和广度，削弱了这一关键安全保障环节的作用。长期依赖 AI 完成标准的安全编码任务（如输入验证），可能会导致团队整体核心安全技能的“退化”。如果开发者逐渐失去独立识别和修复复杂安全问题的能力，组织将形成对 AI 工具的危險路径依赖，从根本上削弱其安全纵深防御能力和技术韧性。

3.4 小结

本章系统剖析了 AI 代码生成技术的双重性。其在提升效率的同时，伴随着由“代码幻觉”和“知识截断”引发的主动漏洞注入风险，以及交互过程中的数据泄露隐患。更深远的间接风险则涉及知识产权合规性与开发者核心安全技能的退化。这些多维度的风险因素共同揭示了 AI 辅助编程面临的核心困境：在自动化程度提升的同时，安全可控性正面临前所未有的挑战。

4 AI 生成代码趋势与特点

4.1 时间发展趋势

纵观 AI 代码生成技术在开源社区的落地进程，我们追踪发现，这并非是一个简单的线性渗透过程，而是一场充满波折、自我修正与范式重构的复杂进化（图2）。AI 并没有像早期的乐观预测那样迅速且无缝地接管软件开发，相反，它与人类开发者的互动关系经历了从“工具崇拜”到“信任危机”，最终迈向“理性共生”的辩证发展阶段。这一演进轨迹不仅映射了模型能力的迭代，更深刻反映了软件工程行业在面对颠覆性技术时的适应与重塑能力。

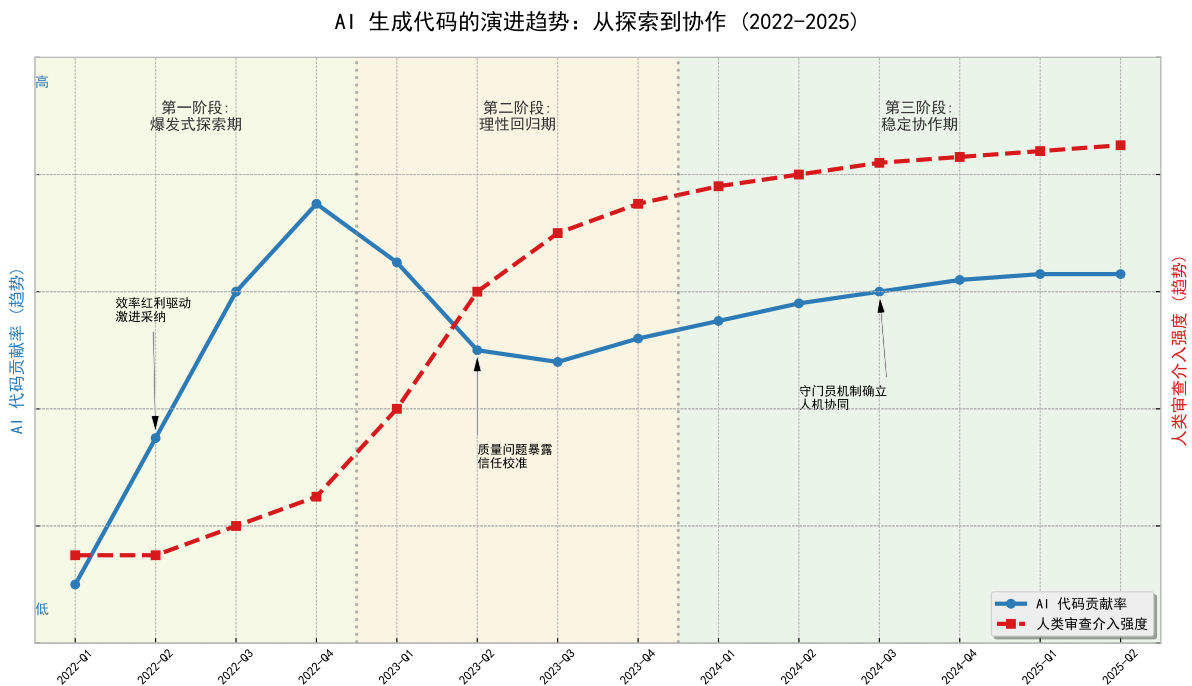


图 2: 代码生成中 AI 的使用趋势

在 AI 辅助编程工具大规模进入市场的初期，开源社区经历了一段显著的“爆发式探索期”。这一阶段的核心特征是“广度优先”的激进采纳。得益于 IDE 插件的无缝集成和极低的使用门槛，AI 技术迅速跨越了早期采用者的鸿沟，在极短的时间内实现了惊人的渗透率增长。此时的开发者深受新技术带来的效率红利驱动，往往抱有一种探索甚至猎奇的心态，倾向于在各类编程任务中广泛尝试使用 AI 生成内容。无论是简单的函数补全、样板代码生成，还是复杂的算法实现，AI 的建议往往被不加甄别地接受并提交。数据流向显示，这一时期 AI 生成代码在新增代码总量中的贡献占比呈现出陡峭的上升曲线，反映出行业对于“自动化编程”概念的极高期待与热情，AI 被视为一种能够立竿见影提升生产力的“银弹”。

然而，随着应用深度的增加，这种早期的狂热不可避免地遭遇了现实工程复杂度的

阻击，行业随之进入了一个漫长而痛苦的“理性回归与信任校准期”。在这一阶段，实证数据曲线出现了明显的震荡、停滞甚至回落，这并非技术的倒退，而是行业认知的成熟。随着代码库规模的扩大，AI 生成代码在长上下文理解上的局限性、对特定领域业务逻辑的误解、以及潜在的非功能性缺陷开始集中暴露。开发者逐渐意识到，虽然 AI 能在几秒钟内生成数百行代码，但后续的调试、审查和维护成本可能远超预期。这种“技术负债”的显现导致了“自动化偏见”的消退，开发者开始从盲目采纳转向审慎评估，有意识地收缩 AI 的使用范围。数据表明，在这一时期，对于高风险、高复杂度或涉及核心架构的场景，AI 的介入率出现了结构性的回调，这标志着开发者正在重新确立人类在代码质量控制中的主导地位。

进入近期阶段，我们观察到 AI 代码的演进已穿越了失望的低谷，迈入了一个全新的“稳定协作与生态融合期”。尽管 AI 代码的生成总量在经历调整后重新回升并趋于饱和，但其背后的生产关系已发生了质的蜕变。此时的 AI 不再被视为试图取代开发者的“独立承包商”，而是被重新定义为高吞吐量的“智能助手”。实证分析揭示了一个关键的生态位变化：人类开发者在人机回路中的角色权重发生了根本性转移——从单纯的代码编写者转型为高阶的审查者与架构师。在这种成熟的协作范式下，AI 与人类的分工边界变得前所未有的清晰：AI 被指派负责那些高重复性、模式化强、且易于验证的任务，如单元测试编写、文档生成、接口适配以及基础的“胶水代码”实现；而人类则收束精力，专注于处理那些 AI 难以触及的领域，如复杂的系统设计、模糊需求的澄清、核心业务逻辑的决策以及最终的安全把关。特别值得注意的是，数据趋势还揭示了 AI 在“自我修复”领域的崛起——越来越多的代码变更不再仅仅是新功能的生成，而是 AI 针对既有代码进行的优化与缺陷修复。这种趋势表明，AI 正在从软件开发的“上游”向“全生命周期”渗透，不仅负责“生”，也开始负责“养”。综上所述，AI 代码生成技术在开源世界的演进，是一部从“狂野生长”走向“精耕细作”的历史。时间轴上的每一次波动，都代表了行业对 AI 能力边界的一次精准测绘。当前形成的这种“AI 负责高效率产出，人类负责高标准把关”的动态平衡，或许正是未来相当长一段时间内软件工程领域的标准常态。这表明，技术进步并没有简单地剔除人类，而是迫使我们进化出了更高级的鉴别能力与系统思维，以驾驭这股强大的自动化洪流。

4.2 编程语言分布

AI 代码生成技术的渗透程度并非在所有技术栈中均匀分布，而是呈现出显著的层次特征（图3）。这种分布态势主要由两个核心因素驱动：一是开源训练语料的丰富度，二是语言本身的语法冗余度。在宏观层面，Python、JavaScript 和 TypeScript 等拥有海量开源生态的高资源语言占据了 AI 生成代码的绝对主力地位。这些语言的语法灵活且容错率较高，使得 AI 能够轻松生成大段业务逻辑。紧随其后的是 Java 和 Go 等企业级语言，虽然其逻辑严谨，但由于存在大量样板代码，开发者倾向于利用 AI 消除重复劳动。相比之下，Rust、C++ 等系统级语言由于对内存安全和生命周期的严格限制，AI 生成代码的采纳率相对较低，开发者在这些场景下往往保持着更审慎的人机协作距离。

这种宏观的渗透率分布为我们理解安全风险提供了背景：高渗透率往往意味着潜在的高风险暴露面。然而，当我们把视角从生成总量切换到漏洞修复这一特定高危场景时，情况发生了微妙的变化。

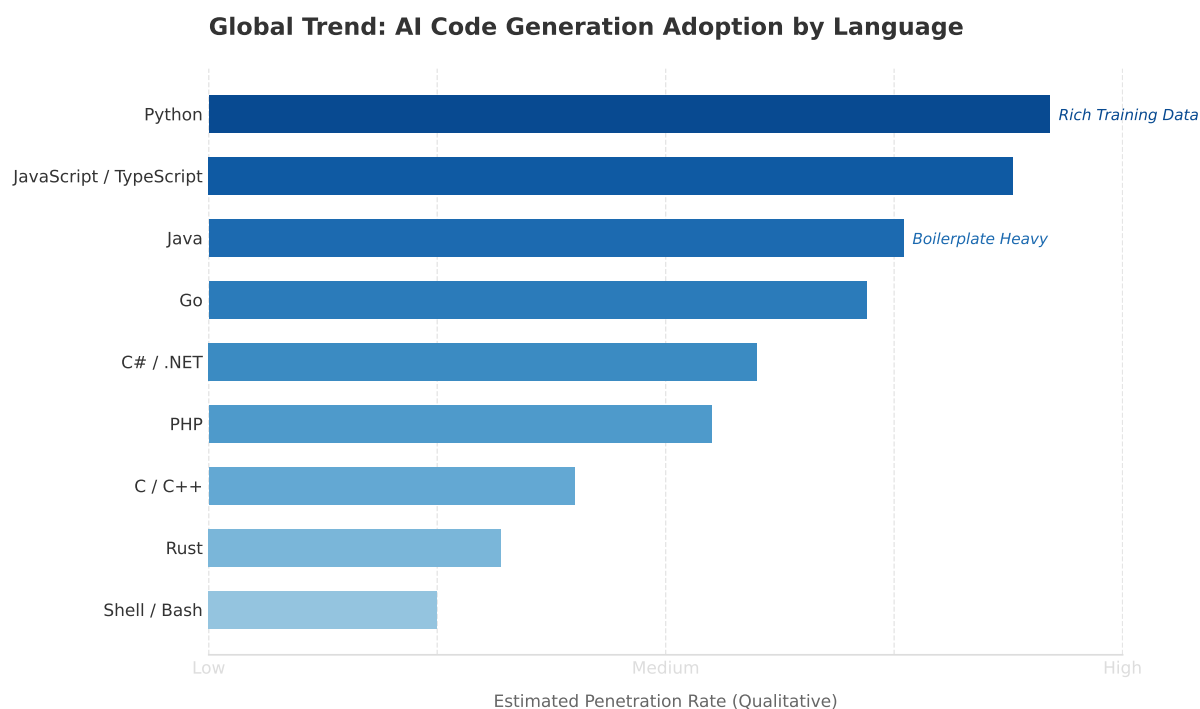


图 3: AI 生成代码在不同编程语言中的相对比例

对多种编程语言和配置文件类型的分析表明（图4），在经历漏洞修复后，绝大多数语言的 AI 代码率呈现出普遍上升的趋势。这一现象与仓库层面的观察结果相符，进一步印证了 AI 辅助编程工具在软件维护周期中日益增长的影响力。无论是在底层系统语言、通用高级语言，还是在前端脚本和标记语言中，AI 代码的占比普遍增加，这表明开发者在处理代码缺陷和迭代功能时，广泛利用 AI 工具来加速开发过程。

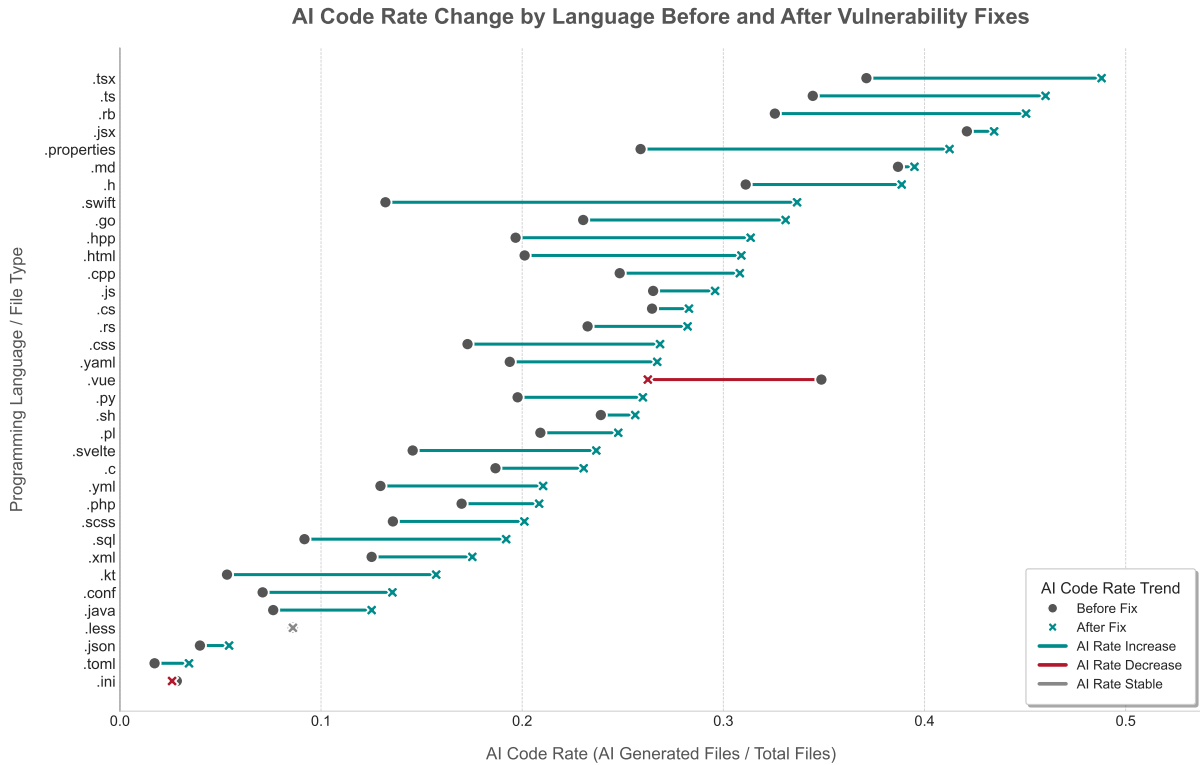


图 4: 漏洞修复前后不同编程语言的 AI 率分布

然而，这种增长并非同质化的。不同类型的编程语言和文件，其 AI 代码率的增长幅度存在显著差异，反映了 AI 技术在不同开发范式中的渗透深度和应用模式。例如，在一些被广泛用于构建动态应用或包含大量样板代码的语言中，AI 代码率的增长尤为明显，这可能说明 AI 工具在这些领域提供了更为直接和高效的辅助。这种差异化渗透，也可能与 AI 模型对特定语言的训练数据量、语言本身的结构复杂性以及社区对 AI 工具的采纳程度等因素有关。

尽管 AI 代码率普遍上升，但值得关注的是，在少数特定的编程语言或配置文件类型中，其 AI 代码率在漏洞修复后却保持稳定甚至出现了下降。这种下降趋势提供了一个至关重要的安全信号：它表明在这些语言类型中，AI 生成代码可能直接被识别为漏洞的贡献者。当这些缺陷被清除时，相关的 AI 代码段也随之被移除或替换，其在整体代码库中的比例随之降低。

不同编程语言的 AI 代码率在漏洞修复后普遍上升，反映了 AI 工具在开发维护中的广泛应用。然而，少数语言 AI 代码率的下降，清晰指示了 AI 生成代码在特定编程范式下可能直接导致漏洞，强调了 AI 代码安全性评估需深入到语言层面，以应对其差异化的潜在风险。

4.3 小结

本章从时间演进轨迹与编程语言分布两个维度，对 AI 代码生成技术在开源生态中的渗透现状进行了深层复盘。数据揭示，行业对 AI 的应用已跨越盲目采纳的狂热期，

进入了人机理性共生的新阶段，确立了“AI 负责高效产出、人类负责关键把关”的协作范式。然而，特定编程语言在漏洞修复后 AI 代码率的结构性下降，客观印证了 AI 生成代码在特定场景下作为“缺陷源头”的风险属性。这种效率红利与潜在隐患并存的复杂局面，表明单纯依赖工具的自动化已触及天花板，软件工程正在向要求更高鉴别力与系统性风控的混合开发模式转型。

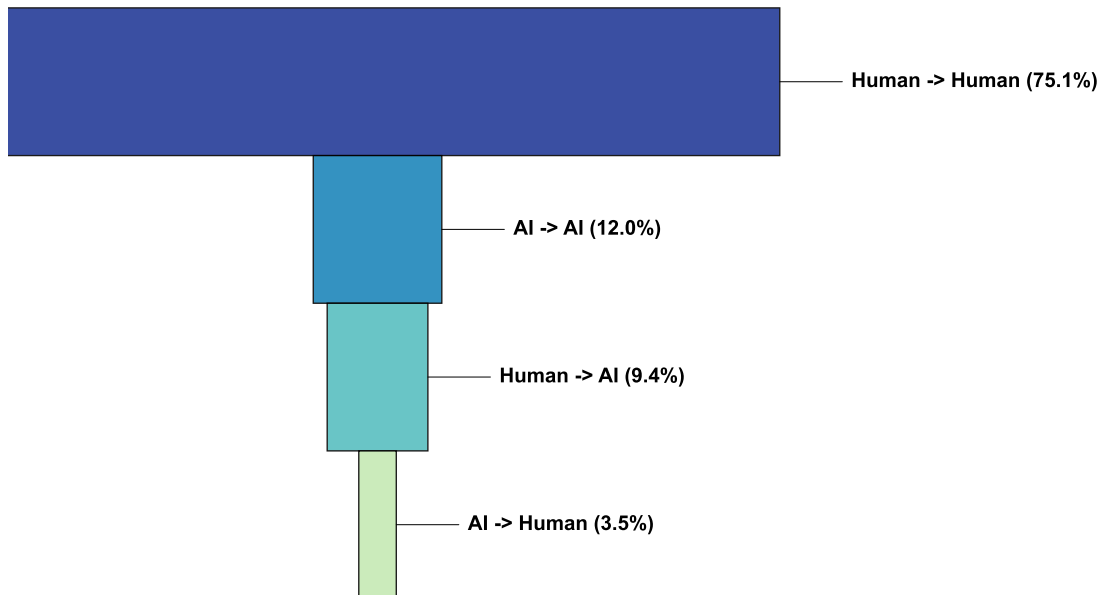
5 AI 生成代码的安全风险与漏洞特征

5.1 漏洞生命周期中 AI 的角色演变

AI 代码生成技术正在对漏洞的生成、修复与演变过程均产生显著影响，本节将从已识别的通用漏洞披露（CVE）视角，深入分析 AI 代码在漏洞修复前后的动态变化，并探讨 AI 生成代码作为潜在漏洞来源或修复工具的复杂角色，揭示 AI 代码与项目安全态势之间的细致关联。

对大量 CVE 数据（图5）的分析后观察到一个关键现象：在部分漏洞的修复过程中，原本由 AI 生成的代码被显著地替换为人工编写的代码。这种“AI 转人工”的模式并非孤立事件，而是在多种 CVE 情境下反复出现。这表明了，这些特定漏洞中，AI 生成代码本身可能直接作为安全缺陷的根本原因。当漏洞被发现并着手解决时，安全工程师和开发者意识到现有 AI 代码存在问题，并选择将其替换为经过人工审查和验证的解决方案。这一发现为本报告前文关于 AI 代码可能引入“代码幻觉”或继承训练数据中不安全模式的风险提供了具象化的证据。它迫使我们审视 AI 代码的内在质量与安全性，尤其是在其被大规模集成到关键业务逻辑中时，可能带来的隐性风险。

Macro-Analysis of AI Code Transformation in Vulnerability Fixes



Total Transformations: 1,766

图 5: 漏洞修复前后 AI 与人工情况变化

与上述现象形成对比，另一类显著趋势表明，AI 生成代码在漏洞修复过程中也扮演了积极的辅助和重构角色。在许多 CVE 的修复案例中，大量原本由人工编写的代码在修复后被转换为 AI 生成代码。这种“人工转 AI”的模式，体现了开发者在应对漏洞时，正越来越多地利用 AI 工具来加速代码的重构、生成新的安全特性，或借助其实现更高效的补丁应用。这体现了 AI 在快速迭代和响应安全事件方面的巨大潜力。然而，这种趋势也引发了新的考量：通过 AI 工具对现有代码进行大规模重构，即使是为了修复已知漏洞，也可能在无意中引入新的复杂性，当 AI 模型自身未能完全理解安全上下文或最佳实践时，甚至还会产生新的、更难追踪的缺陷。因此，AI 在修复中的效率优势必须与严格的验证机制并行，以确保修复方案的稳健性。

从 CVE 角度审视 AI 代码的演变，AI 代码在漏洞生命周期中扮演着双重角色：既可能作为漏洞的直接来源而被人工替换，又被广泛用于辅助和加速漏洞修复过程。这种动态交互揭示了 AI 生成代码的质量与安全性并非一成不变，其高效引入的背后隐含着持续的审查和验证需求。未来的安全策略必须全面整合 AI 代码的特性，建立从生成到维护的全生命周期安全治理框架，以应对 AI 驱动开发带来的复杂挑战。

5.2 AI 引入漏洞的特征画像

与人类开发者因复杂的业务逻辑疏忽或需求理解偏差而导致的漏洞不同，AI 生成代码所引入的缺陷表现出强烈的“模式化”特征。这种特征本质上源于大语言模型对训练数据中历史编码模式的机械模仿，而非对安全原则的深层理解。

我们的实证数据（图6）表明，AI 引入的漏洞并非在所有类型中均匀分布，而是呈现出高度的集中趋势，主要聚焦于“输入验证与数据编码”以及“API 的不安全调用”两大领域：首先，在处理涉及上下文安全边界的场景时，AI 表现出明显的短板。例如，在进行数据序列化、格式化输出或处理外部输入流时，AI 往往难以根据当前的上下文环境自动实施正确的转义或清洗策略。这种对安全上下文感知的缺失，使得注入类风险成为 AI 代码中最常见的隐患之一。其次，AI 倾向于复用训练数据中的陈旧实践。由于模型从海量历史代码中学习，它容易在涉及密码学或敏感操作时，调用已被现代安全标准弃用的陈旧算法或哈希函数。AI 这种“依样画葫芦”的生成逻辑，导致其生成的代码虽然在语法上无可挑剔，但在安全最佳实践上却往往滞后于当前标准。值得注意的是，AI 引入的漏洞极少涉及深层的业务逻辑错误。相反，它们更多体现为代码片段层面的、局部的实现瑕疵。这种“浅层但高频”的分布特征表明，尽管 AI 引入了新的风险源，但这些风险在很大程度上是可以通过成熟的静态分析工具与自动化扫描规则进行有效识别和拦截的。

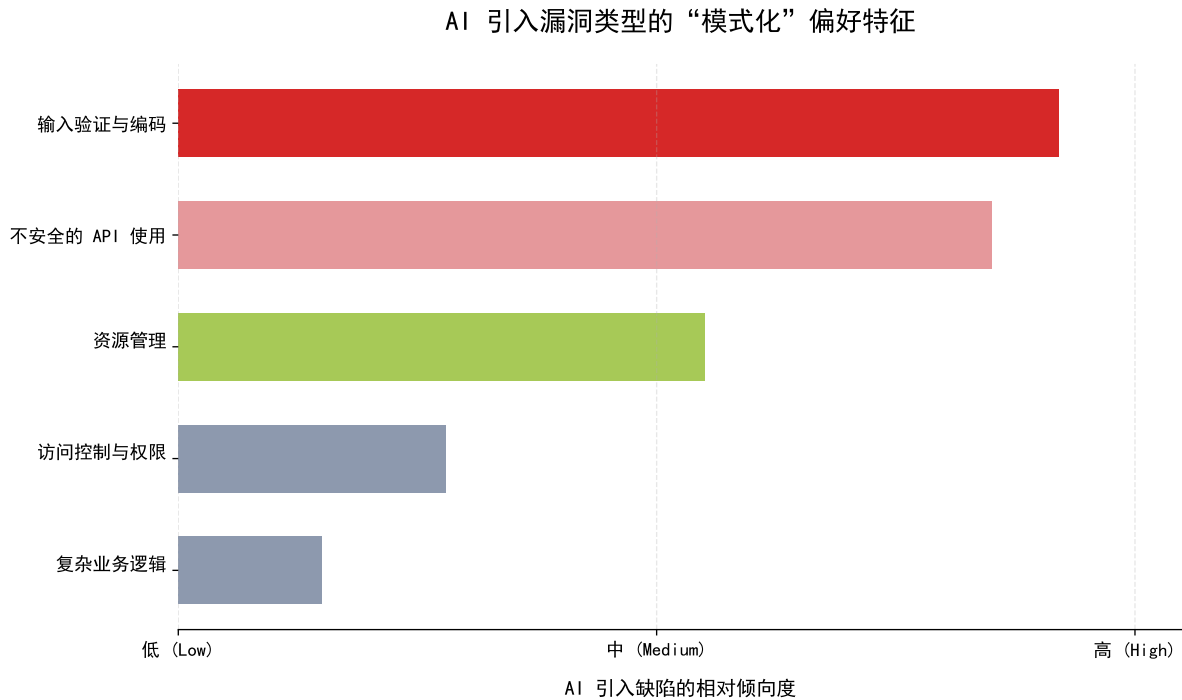


图 6: AI 引入漏洞类型的模式分析

进一步对 AI 引入漏洞的风险特征进行实证研究 (图7) 表明, AI 引入漏洞的风险特征并不“低级”, 而是呈现出“严重度拟人化”与“攻击面网络化”的双重趋势。首先, 在漏洞严重程度维度上, AI 生成代码表现出了与人类开发者惊人的一致性。统计分布显示, AI 引入漏洞的严重等级并没有显著低于人类, 它忠实地“继承”了人类开发者的风险分布模式。这意味着, AI 不仅会犯错, 而且完全有能力引入导致系统瘫痪、数据泄露等后果严重的高危级漏洞。AI 生成的代码在安全性上并不具备天然的“免疫力”或“降级效应”, 其风险上限与人类编写的代码完全持平。其次, 在攻击向量维度上, AI 引入的漏洞展现出了更为危险的结构化偏好——网络化倾向。与主要依赖本地访问或物理接触才能触发的漏洞不同, AI 生成代码中的缺陷更显著地集中在网络攻击向量上。这一趋势在处理 API 接口调用、Web 服务交互及网络协议解析的代码场景中尤为突出。这种分布特征意味着, 由 AI 辅助生成的代码模块, 往往更容易成为远程攻击者的突破口。攻击者无需获取系统的本地权限, 即可通过互联网发送恶意请求来触发这些漏洞。因此, AI 的广泛应用在无形中扩张了软件系统的远程攻击面, 使得系统的边界防御面临更大的压力。这警示我们, 在审查涉及网络通信和对外接口的 AI 生成代码时, 必须提升安全基线标准, 而非仅仅依赖其功能实现的正确性。

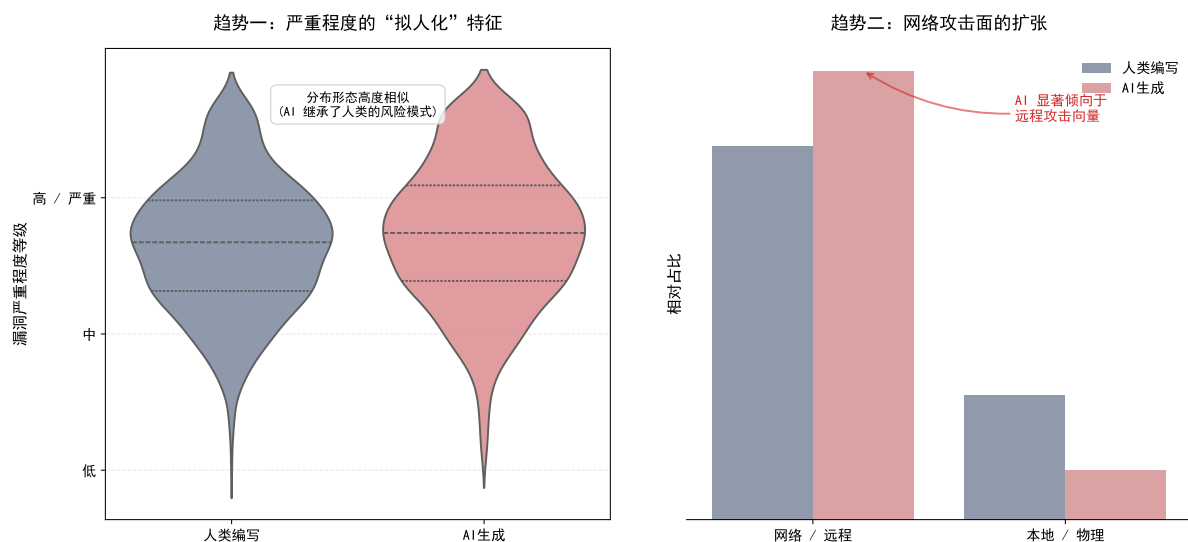


图 7: AI 引入漏洞的风险特征分析

5.3 小结

本章聚焦于漏洞全生命周期，系统剖析了 AI 作为“双刃剑”的安全实质。实证研究表明，AI 既能作为辅助工具加速漏洞修复，亦因其“机械模仿”特性成为输入验证缺失与不安全 API 调用等高危漏洞的直接制造者。其引入的缺陷呈现出“严重度拟人化”与“攻击面网络化”的显著特征，即 AI 生成的漏洞在危害等级上不亚于人类，且更倾向于暴露远程网络攻击面。这一发现打破了 AI 仅产生低级语法错误的刻板印象，警示我们在享受 AI 带来的开发便利时，必须正视其在深层业务逻辑与上下文安全感知上的先天短板，并建立针对性的防御机制。

6 风险缓解建议

面对 AI 代码生成技术所固有的内生缺陷及其引发的供应链复杂安全挑战，仅依赖单一维度的防御手段已显得捉襟见肘。本章建议构建一套全方位、立体化的安全防御体系。如图 8 所示，从评测基准、模型本体安全性以及人机协同治理三个维度协同发力，旨在打造从代码开发到落地的全生命周期安全闭环。

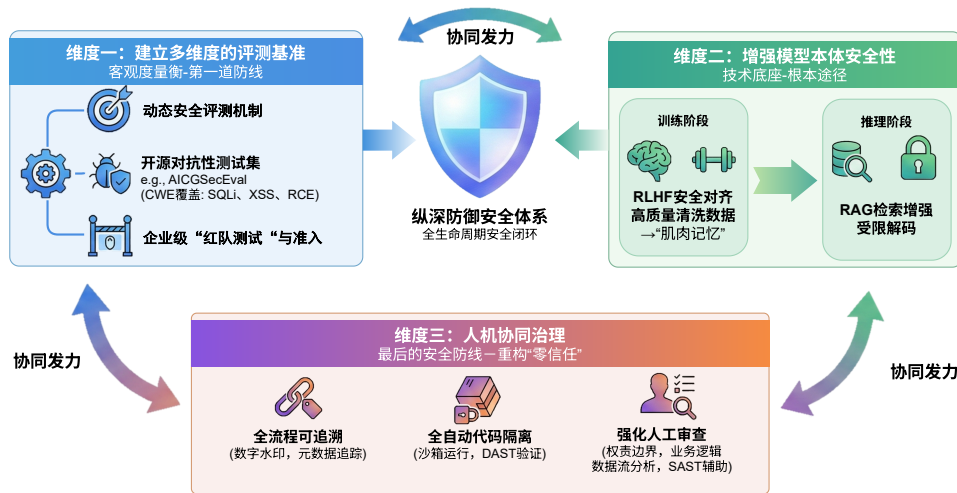


图 8: 风险缓解框架

6.1 建立多维度的评测基准

AI 模型输出的概率性与随机性特征，致使传统的静态测试方法难以有效覆盖其安全边界。因此，必须建立专门针对大语言模型的动态安全评测机制，将其作为模型接入与上线前的第一道防线。行业内目前已有较为成熟的开源实践，例如 A.S.E 等项目，这些工具基于通用缺陷列表（CWE）构建了覆盖 SQL 注入、跨站脚本（XSS）、远程代码执行等高危场景的对抗性测试集，能够量化评估模型在多语言、多场景环境下的生成安全性。企业应当在模型引入阶段建立基于此类成熟基准的“红队测试”与准入机制，通过严格的基线测试与持续的攻防演练，确保上线模型具备足够的鲁棒性，从源头上减少存在原生安全缺陷的模型应用。

6.2 增强模型本体安全性

提升模型本体的安全性是解决代码幻觉、逻辑漏洞与知识截断问题的根本途径，这需要从训练对齐与推理增强两个关键阶段入手。

在训练阶段，应强调源头治理，引入包含安全编码规范的高质量数据集并进行严格的数据清洗。利用人类反馈强化学习（RLHF）技术进行深度的安全对齐，通过惩罚不安全输出、奖励合规代码的机制，引导模型内化安全编程范式，促使模型像人类资深专家一样形成对安全编码的“肌肉记忆”。

在推理阶段，针对模型训练数据的知识滞后性问题，建议采用检索增强生成（RAG）技术，动态挂载最新的 CVE 漏洞库与权威安全公告作为外部知识源，使模型在代码生成过程中具备实时风险规避能力与上下文感知能力。同时，可结合受限解码（Constrained Decoding）技术，从抽象语法树（AST）层面强制约束模型的输出结构，从底层逻辑上阻断非安全 API 的调用与危险代码模式的生成，强制模型输出严格符合语法与安全规范的代码。

6.3 人机协同治理

鉴于当前 AI 技术尚无法实现完全的零误报与零风险，必须在软件供应链流程中重构“零信任”机制，强化人工审查在安全闭环中的关键作用。

建议实施全流程的可追溯管理，对 AI 生成的代码进行强制性的数字水印标记或元数据追踪；对全自动生成的代码（如 AI Agent 产出）默认实行严格的沙箱隔离运行与动态应用程序安全测试（DAST）验证。在治理层面，需重塑适应 AI 辅助编程时代的开发规范，明确“开发者是代码安全最终责任人”的权责边界，防止因过度依赖工具而产生的责任转移。代码审查的重心应从传统的语法错误检查，转向更深层的业务逻辑一致性验证与数据流安全分析，利用静态分析工具（SAST）辅助识别隐蔽缺陷，确保在人机协同模式下，人类工程师始终掌握对软件安全状态的最终裁决权与控制权。

6.4 小结

AI 代码生成的风险治理是一个系统工程。建立评测基准提供了客观的度量衡，增强模型本体安全性夯实了技术底座，而人机协同治理则构筑了最后的安全防线。三者相辅相成，共同构成了一个“纵深防御”的安全体系。唯有通过技术手段与管理流程的深度融合，才能在释放 AI 生产力的同时，有效遏制其潜在的安全风险，推动软件工程向更高效、更安全的方向演进。

7 总结

随着大型语言模型（LLM）的深度应用，人工智能（AI）代码生成技术在显著提升软件开发效率的同时，也重塑了软件供应链的安全边界。AI 作为软件供应链中新兴的“代码贡献者”，其内在的“代码幻觉”和“知识截止日期”特性，可直接导致含有逻辑缺陷或已知漏洞（CVEs）的代码被注入到项目中。此外，与 AI 服务的交互过程中敏感数据泄露的风险，以及生成代码的来源的模糊性，构成了严峻的合规与知识产权挑战。

实证分析进一步揭示，AI 在漏洞生命周期中扮演着双重角色：它既是效率工具，被广泛用于加速漏洞修复和代码重构；但在部分场景下，AI 生成的代码本身就是漏洞的直接来源，在修复过程中被人工代码替换。这种双重性凸显了过度依赖 AI 可能侵蚀开发团队的安全文化，导致“自动化偏见”和核心安全“技能退化”的长期风险。因此，将 AI 安全地集成到开发流程中，需要建立一个超越传统代码审计的综合性治理框架，将开发者从纯粹的“创作者”转变为对人机协作产出具有高度批判性审查能力的“验证者”。

针对上述挑战，本报告提出构建“评测基准-模型安全-协同治理”三位一体的纵深防御体系。这要求在引入阶段建立动态安全评测与红队测试机制；在模型层面通过 RLHF 对齐与 RAG 检索增强提升内生安全性；在应用层面重构“零信任”的人机协同模式，落实代码溯源与沙箱隔离。通过技术增强与管理重塑的深度融合，将开发者从单纯的“创作者”转变为具备高度批判性的“验证者”，确保软件供应链在智能化时代的韧性与安全。